DSA5208: Scalable Distributed Computing for Data Science

Lecture 3: Consistency Models

Zhenning Cai

Department of Mathematics, National University of Singapore

AY2026/2027, Semester 1

1 Motivation

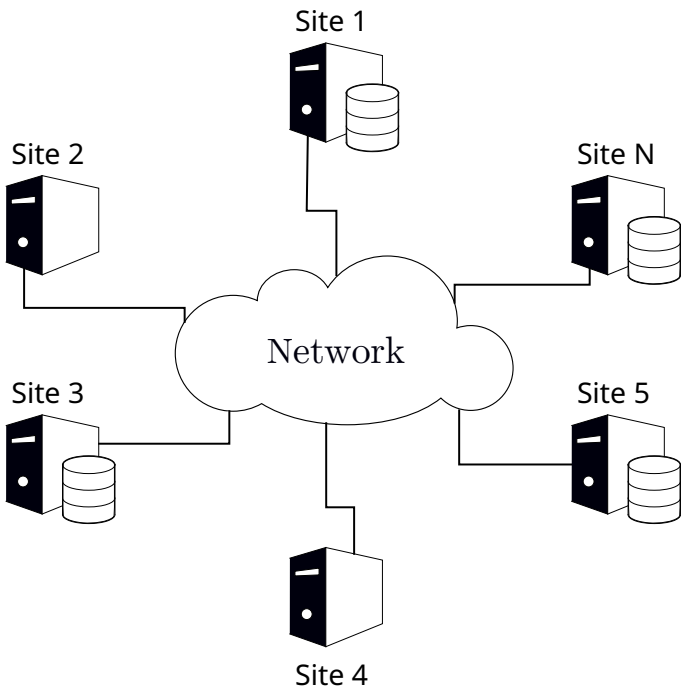
2 Data-centric consistency models

3 Client-centric consistency models

4 Summary

Distributed database

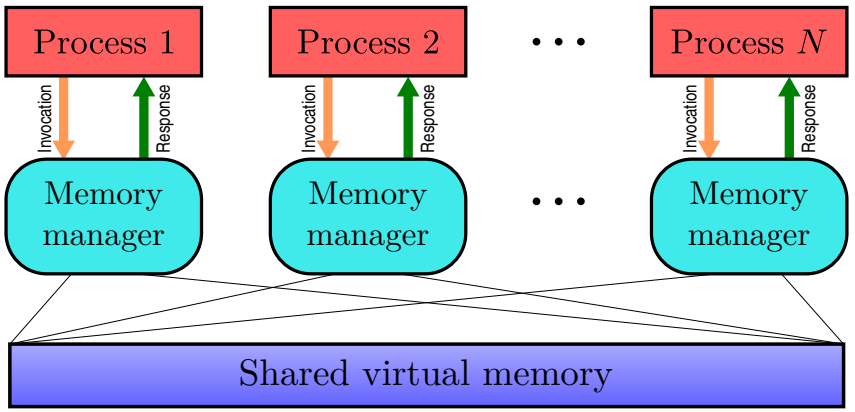
A distributed database is a database in which data is stored across different physical locations.



- The database may be dispersed over a network of interconnected computers at different physical locations.
- The data may be replicated for better reliability and performance.
- Consistency must be maintained between data on different sites.

Distributed shared memory

Distributed shared memory is a form of memory architecture where physically separated memories can be addressed as a single shared address space.

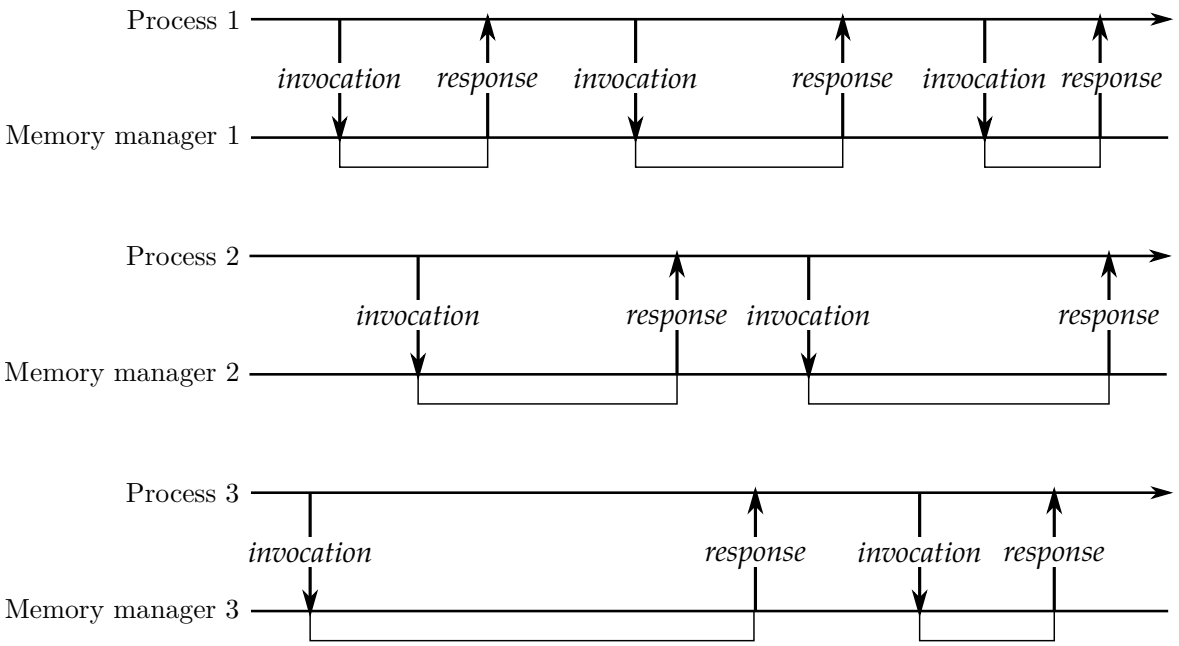


Full-replication algorithm:

- It replicates all data on each host.
- Read requests are serviced locally, requiring no communication overhead.
- Write requests are sent to all hosts so that all copies can be updated.

Invocations and responses

On each process, any two pairs of invocations and responses do not overlap.

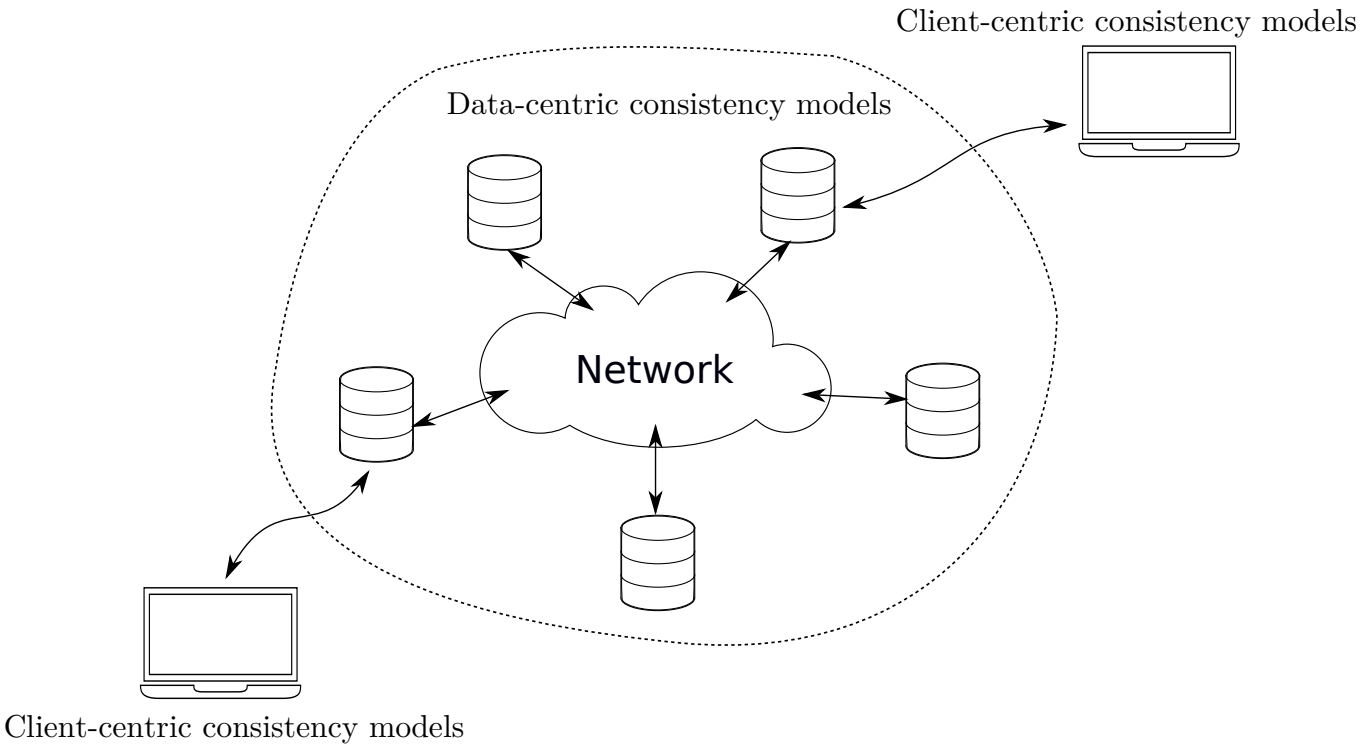


Example:

- Invocation: *Read* x
  Response: $x = 4$
- Invocation: *Write* 8 *to* x
  Response: *Done*

Consistency model

A consistency model refers to a set of rules or constraints that dictate how shared data should appear to processes in or communicating with a distributed system.



1 Motivation

2 Data-centric consistency models

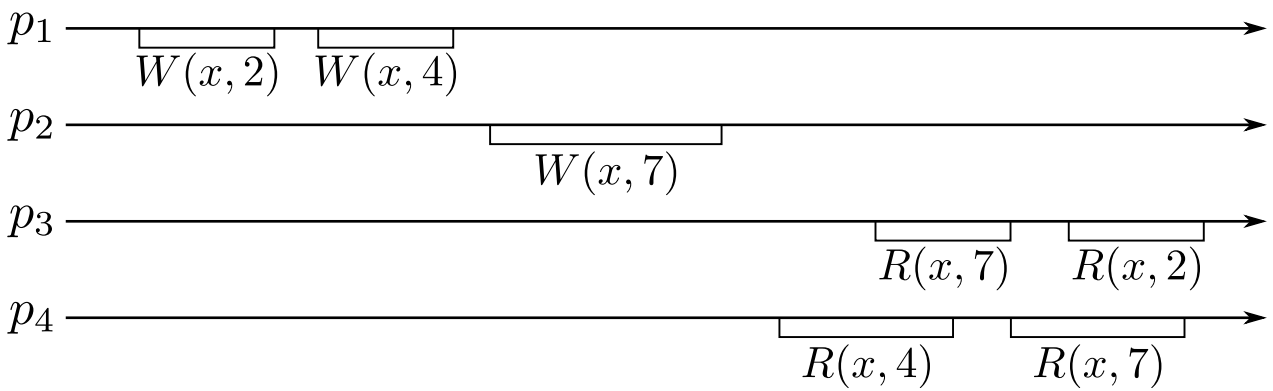
3 Client-centric consistency models

4 Summary

Data-centric consistency models

Data-centric consistency models define rules to ensure that all processes in the system agree on the state of the shared data.

Example:



$W(x,a)$: Write value a to variable x $R(x,a)$: Read variable x and get value a

- Allowed by causal consistency.
- Not allowed by strict consistency or sequential consistency.

1 Motivation

2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

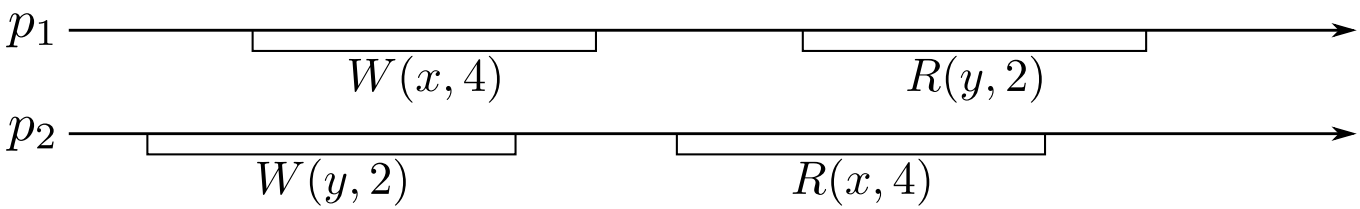
4 Summary

Strict consistency (linearizability)

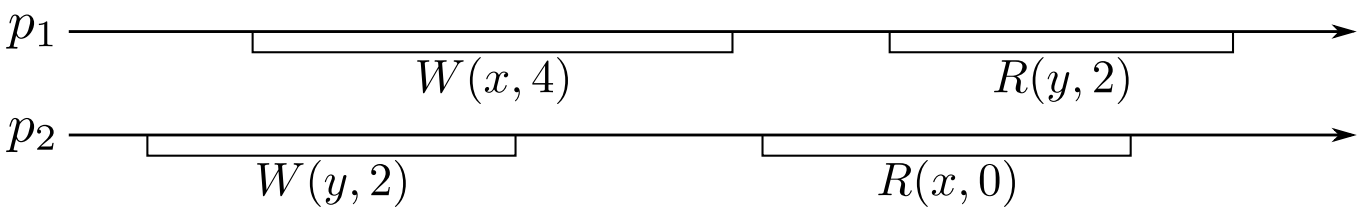
Strict consistency (also called linearizability) requires every write by any process be seen instantaneously by all processors.

Examples: Initial values of x and y are both zero.

- Linearizable:



- Not linearizable:



Definition of linearizability

Definition of linearizability: All *Read* and *Write* operations on all processes can form a sequence such that

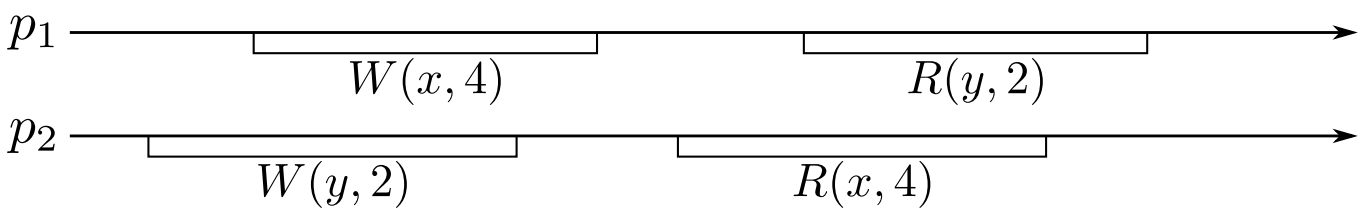
1. For any variable v, every $R(v, \cdot)$ returns the value of the most recent $W(v, \cdot)$ in the sequence.
2. If the response of operation $op1$ occurs before the invocation of operation $op2$, then $op1$ occurs before $op2$ in the sequence.

Definition of linearizability

Definition of linearizability: All *Read* and *Write* operations on all processes can form a sequence such that

1. For any variable v, every $R(v, \cdot)$ returns the value of the most recent $W(v, \cdot)$ in the sequence.
2. If the response of operation $op1$ occurs before the invocation of operation $op2$, then $op1$ occurs before $op2$ in the sequence.

Example:



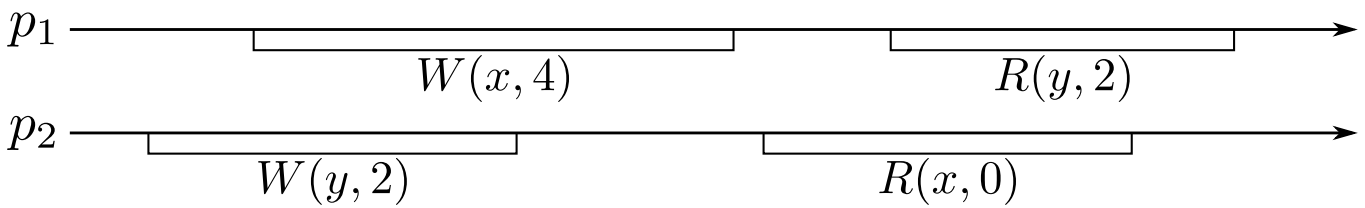
Sequence: $W(x, 4)$, $W(y, 2)$, $R(y, 2)$, $R(x, 4)$

Definition of linearizability

Definition of linearizability: All *Read* and *Write* operations on all processes can form a sequence such that

1. For any variable v, every $R(v, \cdot)$ returns the value of the most recent $W(v, \cdot)$ in the sequence.
2. If the response of operation $op1$ occurs before the invocation of operation $op2$, then $op1$ occurs before $op2$ in the sequence.

Example:



$R(x, 0)$ occurs after $W(x, 4)$ but gets a value before $W(x, 4)$.

Implementation of linearizability

Idea: Use FIFO-total order broadcast to guarantee that all processes see the sequences of *Read*s and *Write*s in the same order.

Algorithm Implementation of linearizability in the memory manager

```
1:  on receiving a Read or Write request from application do
2:      Send a FIFO-total order broadcast of the request to all processes
3:      Await the message that was broadcast
4:      if the request is Read then
5:          Return the value of the local replica
6:      else if the request is Write then
7:          Write the value to the local replica and return
8:      end if
9:  end on

10: on receiving a Write request from network do
11:     Write the value to the local replica
12: end on

13: No operations needed when receiving a Read request from network
```

1 Motivation

2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

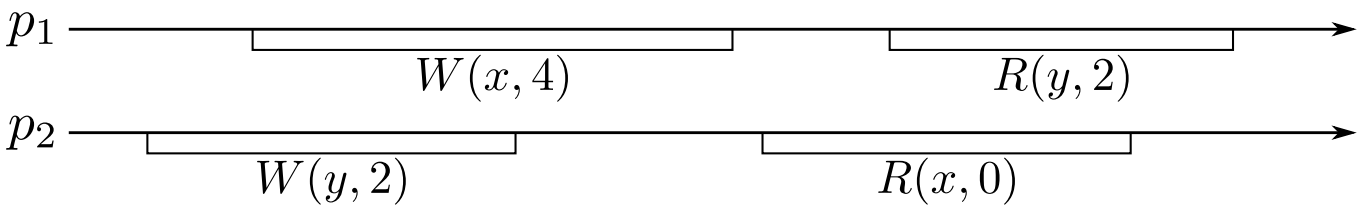
4 Summary

Sequential consistency

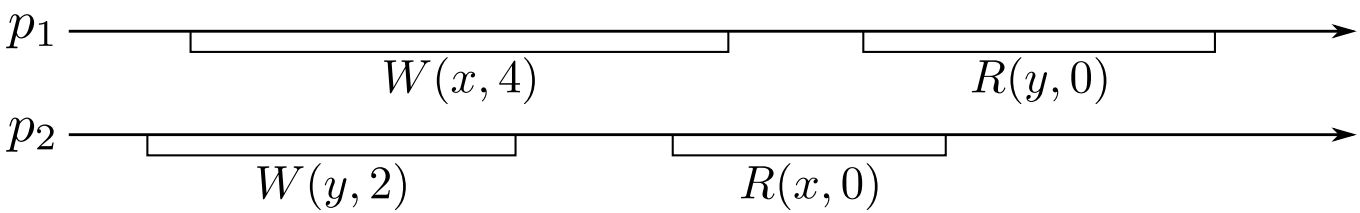
- Linearizability is expensive and difficult to implement.
- Sequential consistency is a weaker model that only requires writes to variables be seen in the same order by all processes.

Examples: Initial values of x and y are both zero.

- Sequentially consistent:



- Not sequentially consistent:



Definition of sequential consistency

Definition of sequential consistency: All *Read* and *Write* operations on all processes can form a sequence such that

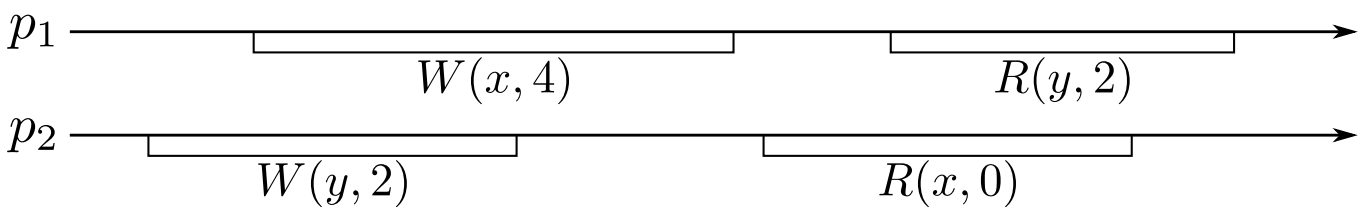
1. For any variable v, every $R(v, \cdot)$ returns the value of the most recent $W(v, \cdot)$ in the sequence.
2. If the response of operation $op1$ occurs before the invocation of operation $op2$, and $op1$ and $op2$ are on the same process, then $op1$ occurs before $op2$ in the sequence.

Definition of sequential consistency

Definition of sequential consistency: All *Read* and *Write* operations on all processes can form a sequence such that

1. For any variable v, every $R(v, \cdot)$ returns the value of the most recent $W(v, \cdot)$ in the sequence.
2. If the response of operation $op1$ occurs before the invocation of operation $op2$, and $op1$ and $op2$ are on the same process, then $op1$ occurs before $op2$ in the sequence.

Example:



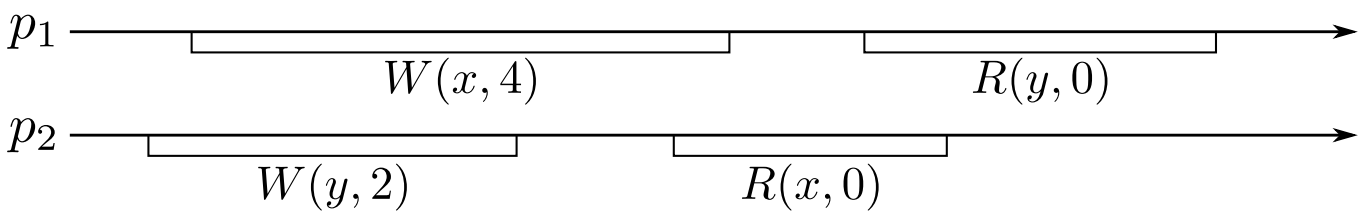
Sequence: $W(y, 2)$, $R(x, 0)$, $W(x, 4)$, $R(y, 2)$

Definition of sequential consistency

Definition of sequential consistency: All *Read* and *Write* operations on all processes can form a sequence such that

1. For any variable v, every $R(v, \cdot)$ returns the value of the most recent $W(v, \cdot)$ in the sequence.
2. If the response of operation $op1$ occurs before the invocation of operation $op2$, and $op1$ and $op2$ are on the same process, then $op1$ occurs before $op2$ in the sequence.

Example:



$W(x, 4)$ should occur after $R(x, 0)$, and therefore happens after $W(y, 2)$. Similarly, $W(y, 2)$ happens after $W(x, 4)$, resulting in a contradiction.

Implementation of sequential consistency

Idea: Since sequential consistency only requires *Write*s to have the same order on all processes, we only need to perform FIFO-total order broadcast for *Write* requests.

Algorithm Implementation of sequential consistency

```
1:  on receiving a Read request from application do
2:      Return the value of the local replica
3:  end on

4:  on receiving a Write request from application do
5:      Send a FIFO-total order broadcast of the request to all processes
6:      Await the message that was broadcast
7:      Write the value to the local replica and return
8:  end on

9:  on receiving a Write request from network do
10:     Write the value to the local replica
11: end on
```

1 Motivation

2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

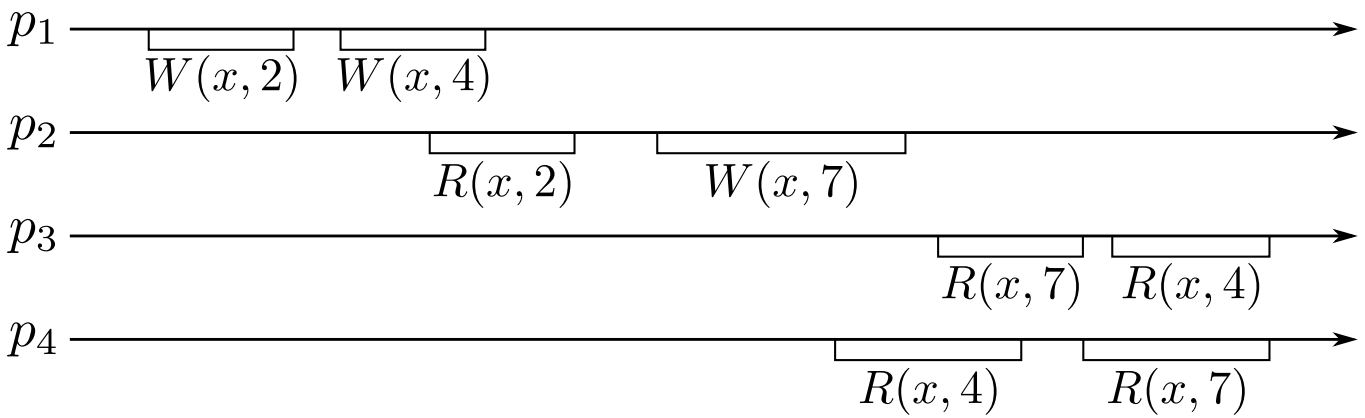
4 Summary

Causal consistency

- Implementation of the FIFO-total order broadcast in linearizability and sequential consistency is expensive.
- Causal consistency only requires *causally related Write* requests to be seen in the same order by all processes.
- The causal precedence between operations is defined by
  1. If two operations $op1$ and $op2$ are on the same process, and the invocation of $op2$ is after the response of $op1$, then $op1$ happens before $op2$.
  2. A *Write* operation happens before a *Read* operation on another process if the *Read* returns the value written by the *Write*.
  3. $op1 \rightarrow op3$ if $op1 \rightarrow op2$ and $op2 \rightarrow op3$.
- **Definition:** For any $R(x, a)$, there exists a $W(x, a)$ happening before it, and there are no writes $W(x, \cdot)$ happening in between.

Examples

Causally consistent:

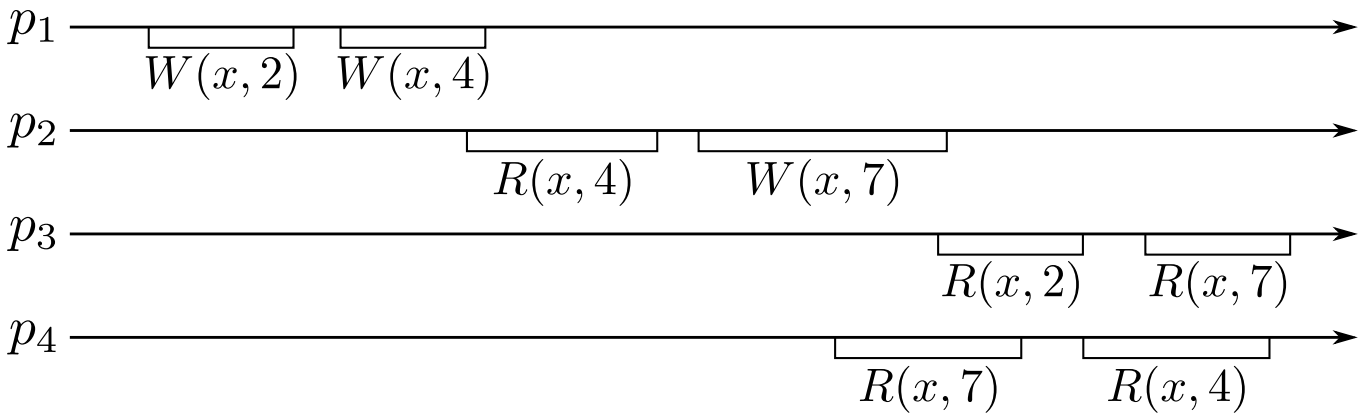


Causal precedences:

$$W(x,2) \rightarrow W(x,4) \rightarrow R(x,4),$$
$$W(x,2) \rightarrow R(x,2) \rightarrow W(x,7) \rightarrow R(x,7).$$

Examples

Not causally consistent:



Invalid causal precedence:

$$W(x,2) \rightarrow W(x,4) \rightarrow R(x,4) \rightarrow W(x,7) \rightarrow R(x,7) \rightarrow R(x,4).$$

Implementation of causal consistency

Idea: Causal consistency requires only causal order for *Write* requests. We just need to replace the FIFO-total order broadcast to the causal order broadcast in the implementation of sequential consistency.

Algorithm Implementation of causal consistency

```
1:  on receiving a Read request from application do
2:      Return the value of the local replica
3:  end on

4:  on receiving a Write request from application do
5:      Send a causal order broadcast of the request to all processes
6:      Await the message that was broadcast
7:      Write the value to the local replica and return
8:  end on

9:  on receiving a Write request from network do
10:     Write the value to the local replica
11: end on
```

1 Motivation

2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

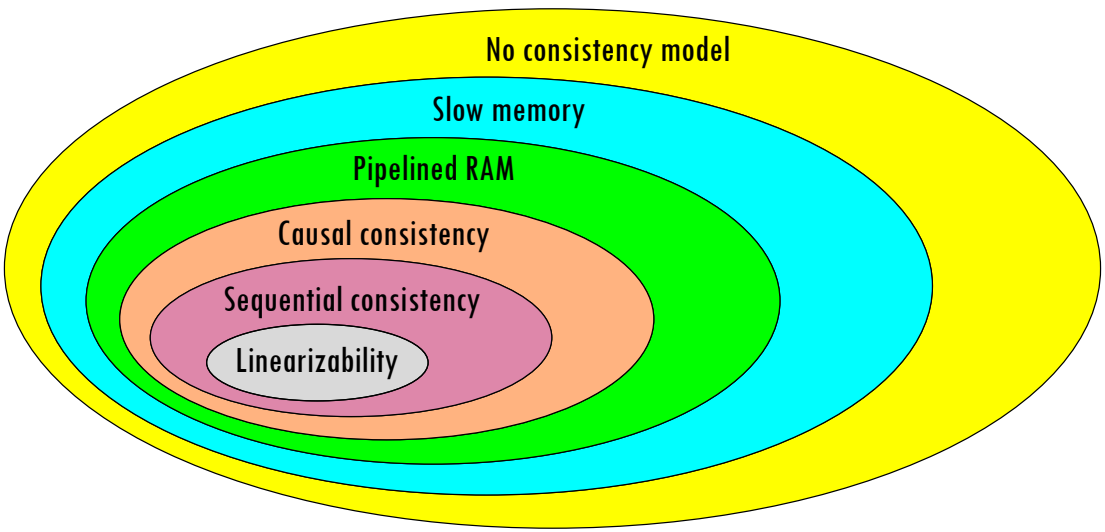
3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

4 Summary

Hierarchy of consistency models

Other weaker consistency models:

- **Pipelined RAM consistency:** requires all *Write* operations issued by the same process to be seen in the same order in all other processes.
- **Slow memory:** requires all *Write* operations issued by the same process to the same variable to be seen in the same order in all other processes.



1 Motivation

2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

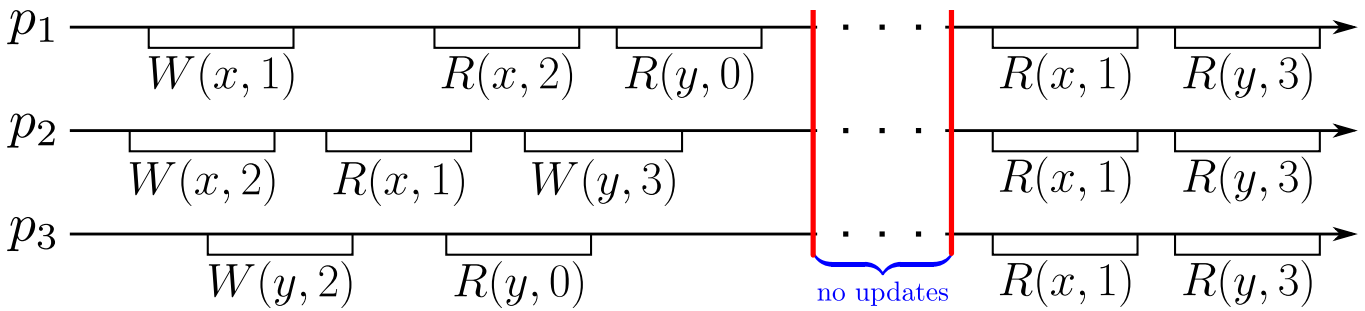
4 Summary

Eventual consistency

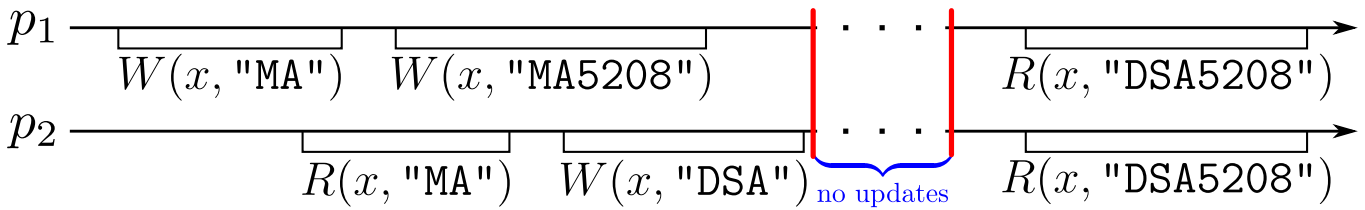
Definition:

If no new updates are made, eventually the replica on all processes will have the same value.

Example 1:



Example 2:



Implementation

- All *Read*s return the local replica immediately.
- On receiving a *Write* request from application, perform a FIFO broadcast of the request to all processes.
- On receiving a *Write* request from network, compare the timestamp of the message with the local timestamp.
  - If one happens before the other, write the newer value to the variable.
  - Otherwise, try to merge the writes. If they are not mergeable, keep both versions and leave the conflict to the users to resolve.

More about conflict resolution: `https://youtu.be/yCcWpzY8dIA`

1 Motivation

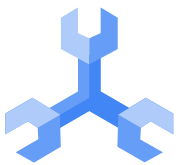
2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

4 Summary

Consistency models in some distributed database systems

Google Cloud Spanner:



Does Spanner provide strong consistency?

Yes. In fact, Spanner provides external consistency, which is a stronger property than strong consistency. The default mode for reads in Spanner is "strong", which guarantees that they observe the effects of all transactions that committed before the start of the operation, independent of which replica receives the read.

What is the difference between strong consistency and external consistency?

A replication protocol exhibits "strong consistency" if the replicated objects are linearizable. Like linearizability, "strong consistency" is weaker than "external consistency", because it does not say anything about the behavior of transactions.

`https://cloud.google.com/spanner/docs/true-time-external-consistency`

Consistency models in some distributed database systems

CockroachDB:



CockroachDB's consistency model: more than serializable, less than strict serializability

Now that we've discussed some general concepts, let's talk about how they apply to CockroachDB. CockroachDB is an open-source, transactional, SQL database and it's also a distributed system. In my opinion, it comes pretty close to being the Holy Grail of databases: it offers a high degree of "consistency", it's very resilient to machine and network failures, it scales well and it performs well. This combination of features already makes it unique enough; the system goes beyond that and brings new concepts that are quite game-changing — good, principled control over data placement and read and write latency versus availability tradeoffs in geographically-distributed clusters. All without ever sacrificing things we informally refer to as "consistency" and "correctness" in common parlance. Also it's improving every day at a remarkable pace. I'm telling you — you need to try this thing!

`https://www.cockroachlabs.com/blog/consistency-model/`

1 Motivation

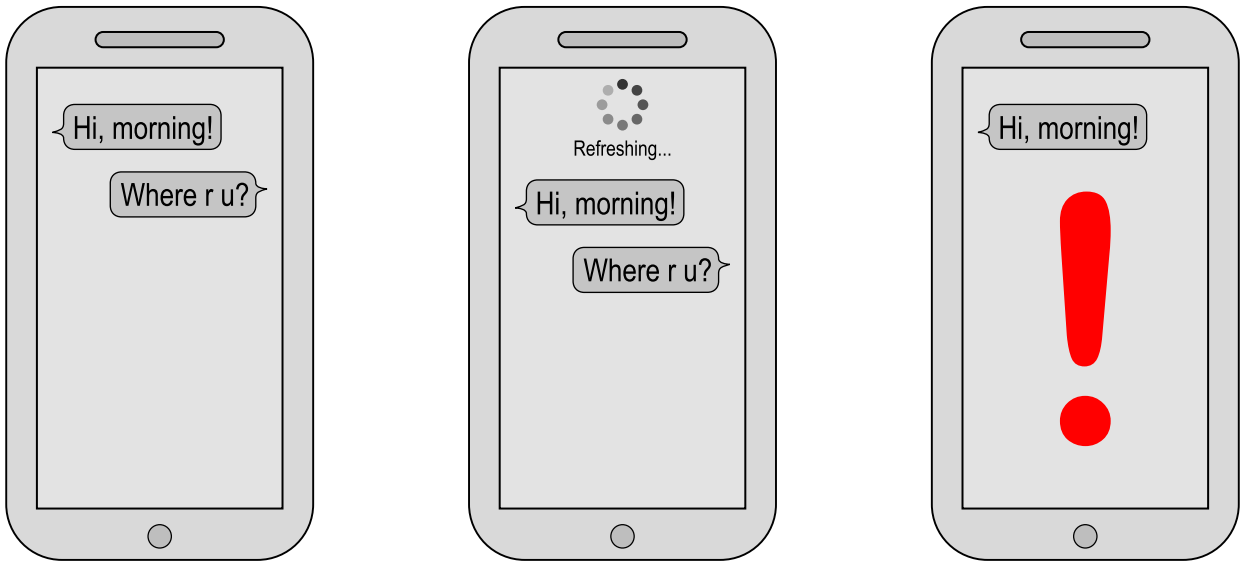
2 Data-centric consistency models

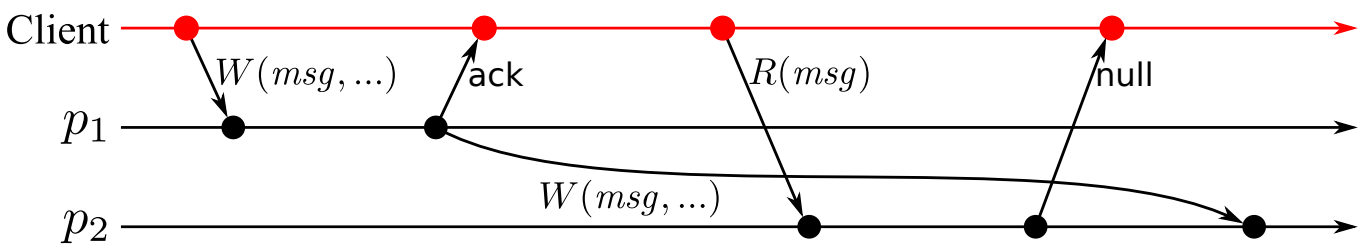
3 Client-centric consistency models

4 Summary

Client-centric consistency models

Client-centric consistency models define rules on what happens when a client reads or writes data on a distributed system.





1 Motivation

2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

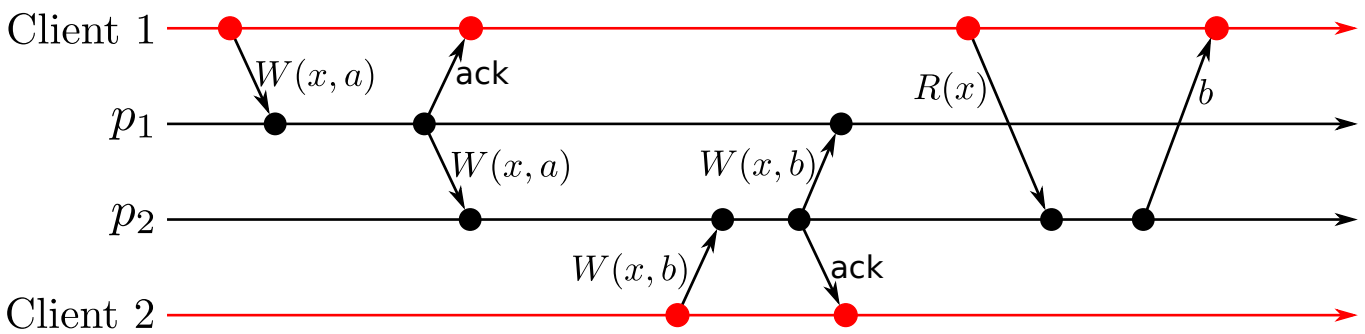
4 Summary

Read-your-writes consistency

Definition:

The effect of a write operation by a process on data item x will always be seen by a successive read operation on x by the same process.

Consistency holds:

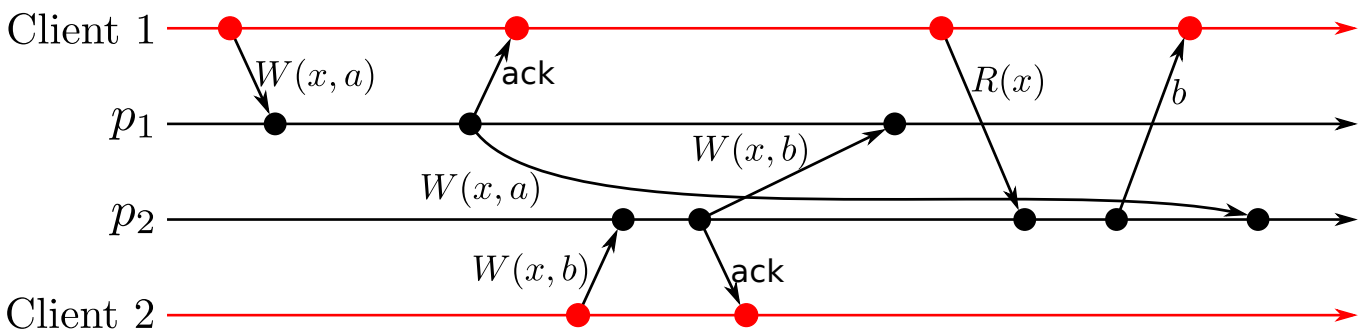


Read-your-writes consistency

Definition:

The effect of a write operation by a process on data item x will always be seen by a successive read operation on x by the same process.

Consistency is violated:

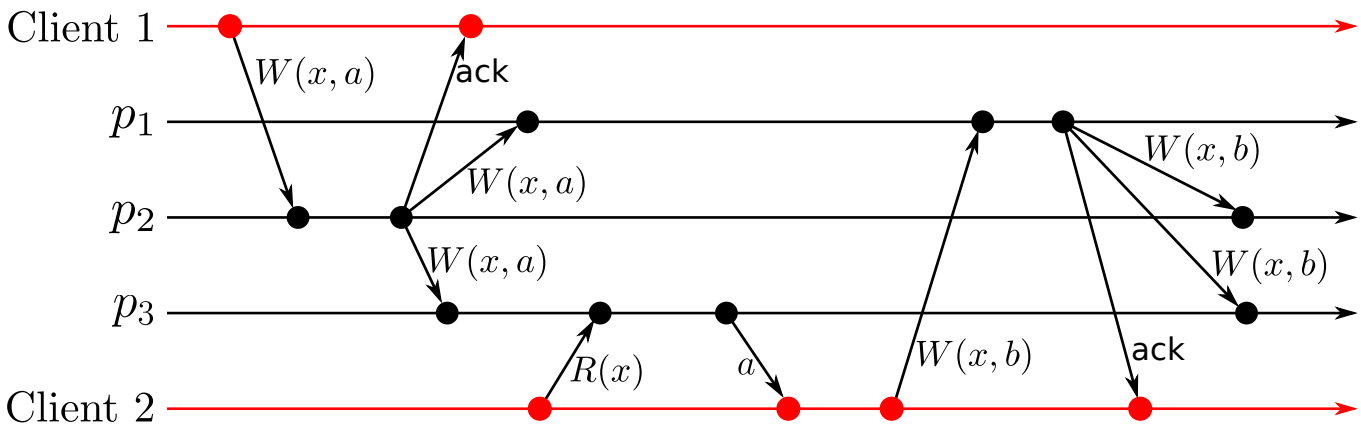


Writes-follow-reads consistency

Definition:

A write operation by a process on a data item x following a previous read operation on x by the same process is guaranteed to take place on the same or a more recent value of x that was read.

Consistency holds:

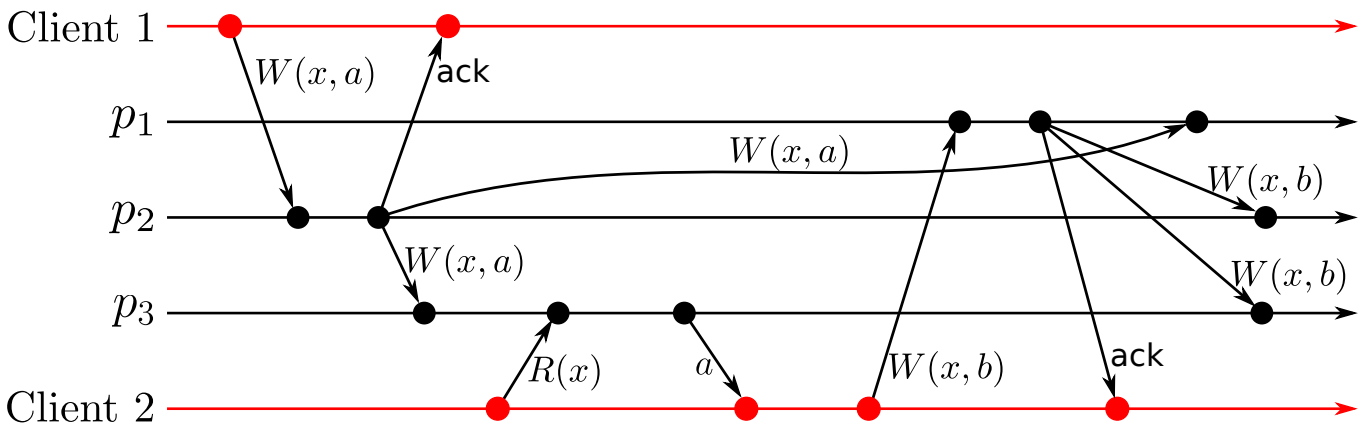


Writes-follow-reads consistency

Definition:

A write operation by a process on a data item x following a previous read operation on x by the same process is guaranteed to take place on the same or a more recent value of x that was read.

Consistency is violated:

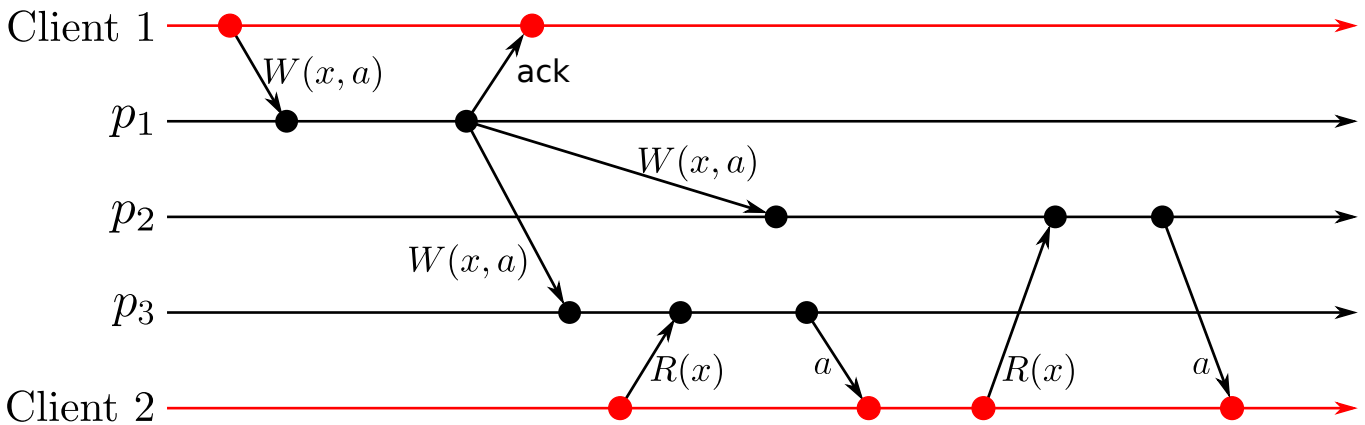


Monotonic-reads consistency

Definition:

If a process reads the value of a data item x, any successive read operation on x by that process will always return that same value or a more recent value.

Consistency holds:

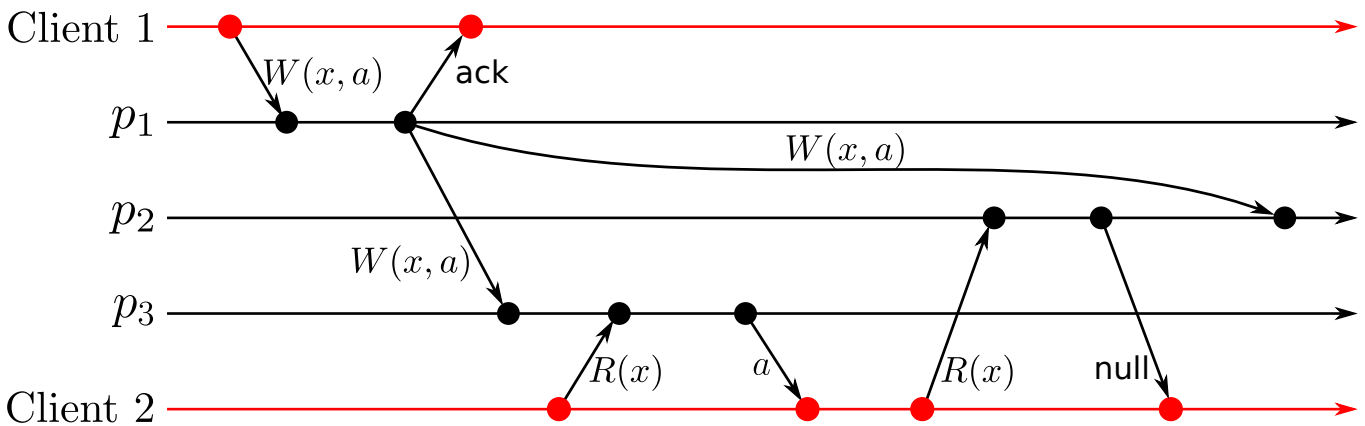


Monotonic-reads consistency

Definition:

If a process reads the value of a data item x, any successive read operation on x by that process will always return that same value or a more recent value.

Consistency is violated:

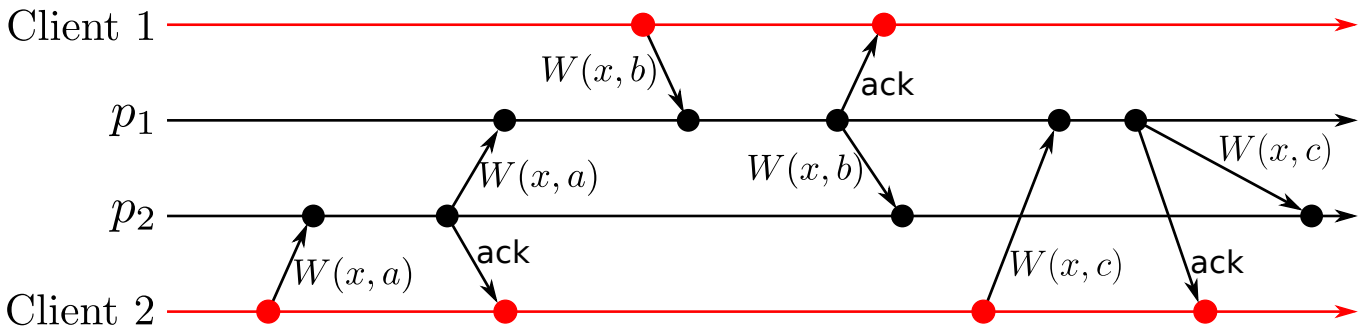


Monotonic-writes consistency

Definition:

A write operation by a process on a data item x is completed before any successive write operation on x by the same process.

Consistency holds:

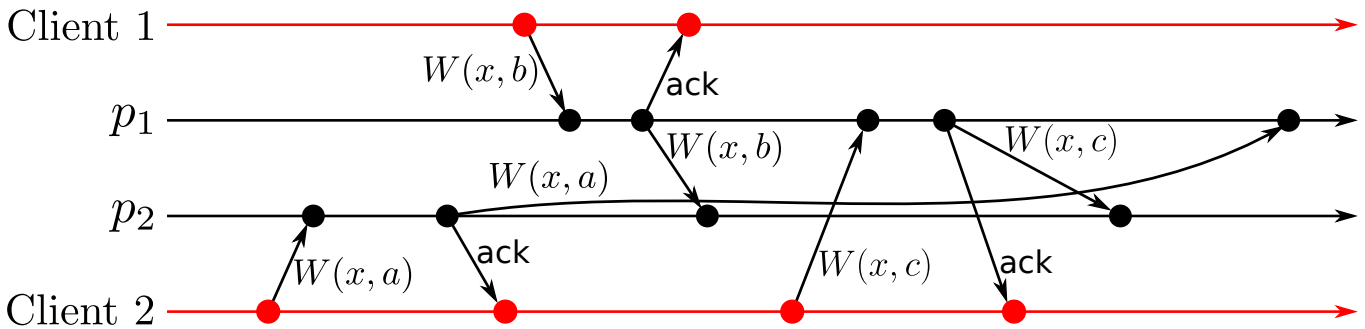


Monotonic-writes consistency

Definition:

A write operation by a process on a data item x is completed before any successive write operation on x by the same process.

Consistency is violated:



1 Motivation

2 Data-centric consistency models
- Strict consistency (linearizability)
- Sequential consistency
- Causal consistency
- Weaker consistency models
- Eventual consistency
- Consistency models in some distributed database systems

3 Client-centric consistency models
- Four consistency models
- Consistency models in some distributed database systems

4 Summary

Consistency models in some distributed database systems

MongoDB:

mongoDB.

Causal Consistency and Read and Write Concerns

With MongoDB's causally consistent client sessions, different combinations of read and write concerns provide different causal consistency guarantees.

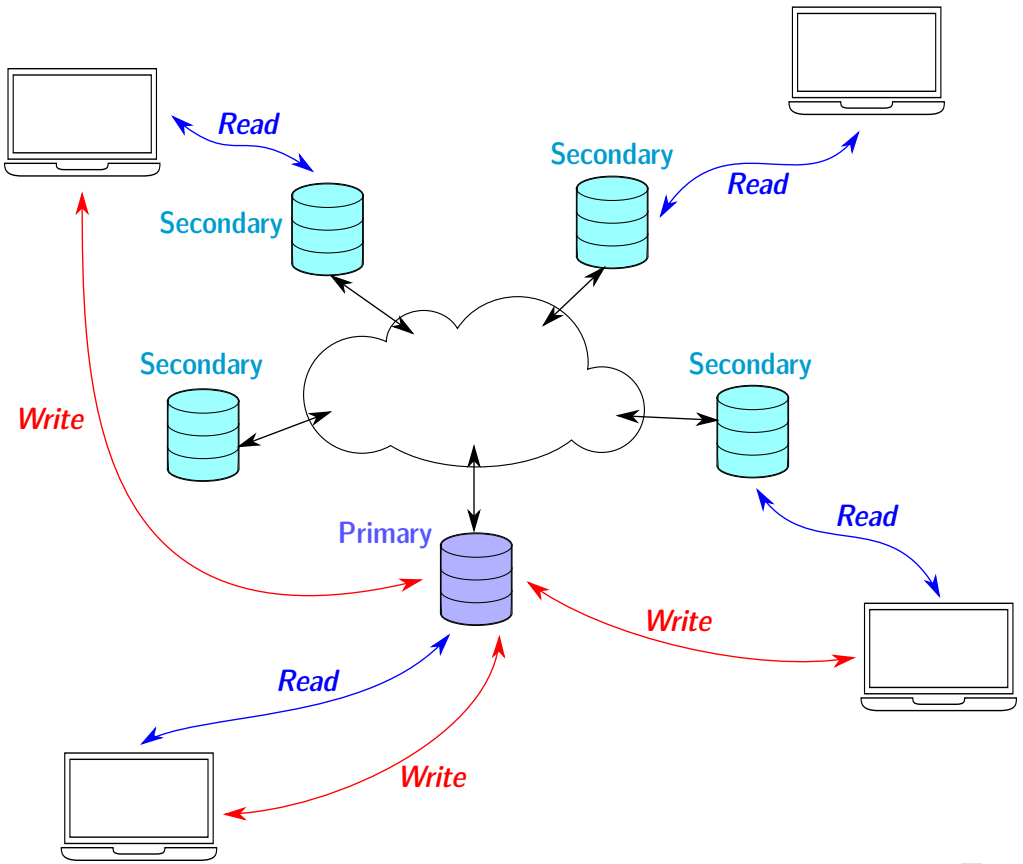
The following table lists the specific guarantees that various combinations provide:

Read Concern	Write Concern	Read own writes	Monotonic reads	Monotonic writes	Writes follow reads
`"majority"`	`"majority"`	✅	✅	✅	✅
`"majority"`	`{ w: 1 }`		✅		✅
`"local"`	`{ w: 1 }`				
`"local"`	`"majority"`			✅	

https://www.mongodb.com/docs/manual/core/causal-consistency-read-write-concerns/

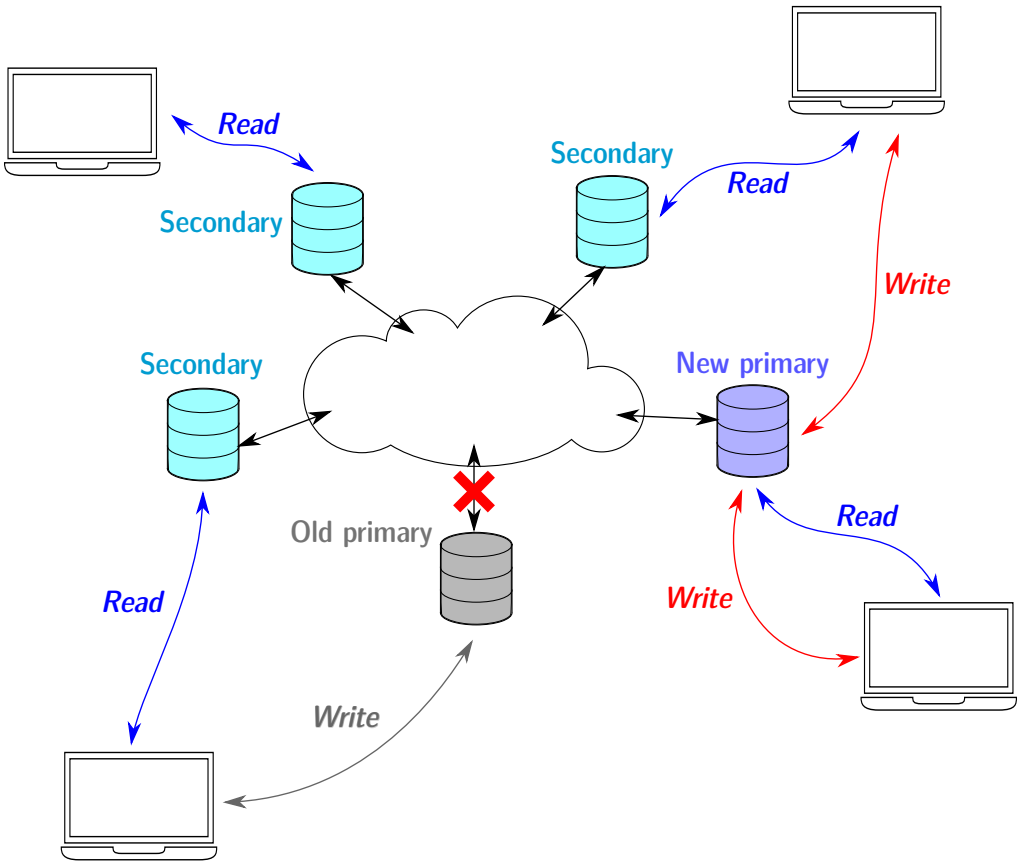
How MongoDB works

- Members of the distributed database system will elect a primary, other nodes are called secondaries.
- All *Write* requests will be sent to the primary, while *Read* requests can be processed by secondaries.



How MongoDB works

- When the primary is down, a new primary will be elected.
- During the transition, *Read* requests can still be serviced, but no *Write* requests will be processed.



Read and write concerns

Read concern:

- `"majority"`: The query returns the data that has been acknowledged by a majority of the replica set members.
- `"local"`: The query returns data from the instance with no guarantee that the data has been written to a majority of the replica set members.

Write concern:

- `"majority"`: The query requests that the *Write* operation has been applied to a majority of the members.
- `"{ w: 1 }"`: The query guarantees that the *Write* operation has propagated to the primary.

Local and majority-committed snapshots

Conceptually, a replica-set member may maintain two views of the data:

- the latest (local) snapshot, containing its newest updates;
- the majority-committed snapshot, containing only updates known to have reached a majority of the replica set.

member	latest snapshot	majority-committed snapshot
Primary	$x = 5$	$x = 3$
Secondary 1	$x = 4$	$x = 3$
Secondary 2	$x = 3$	$x = 3$
Secondary 3	$x = 3$	$x = 3$
Secondary 4	$x = 3$	$x = 3$

The updates $x = 4$ and $x = 5$ are still too new to be majority committed.

Read and write concern "majority"

Write concern "majority":

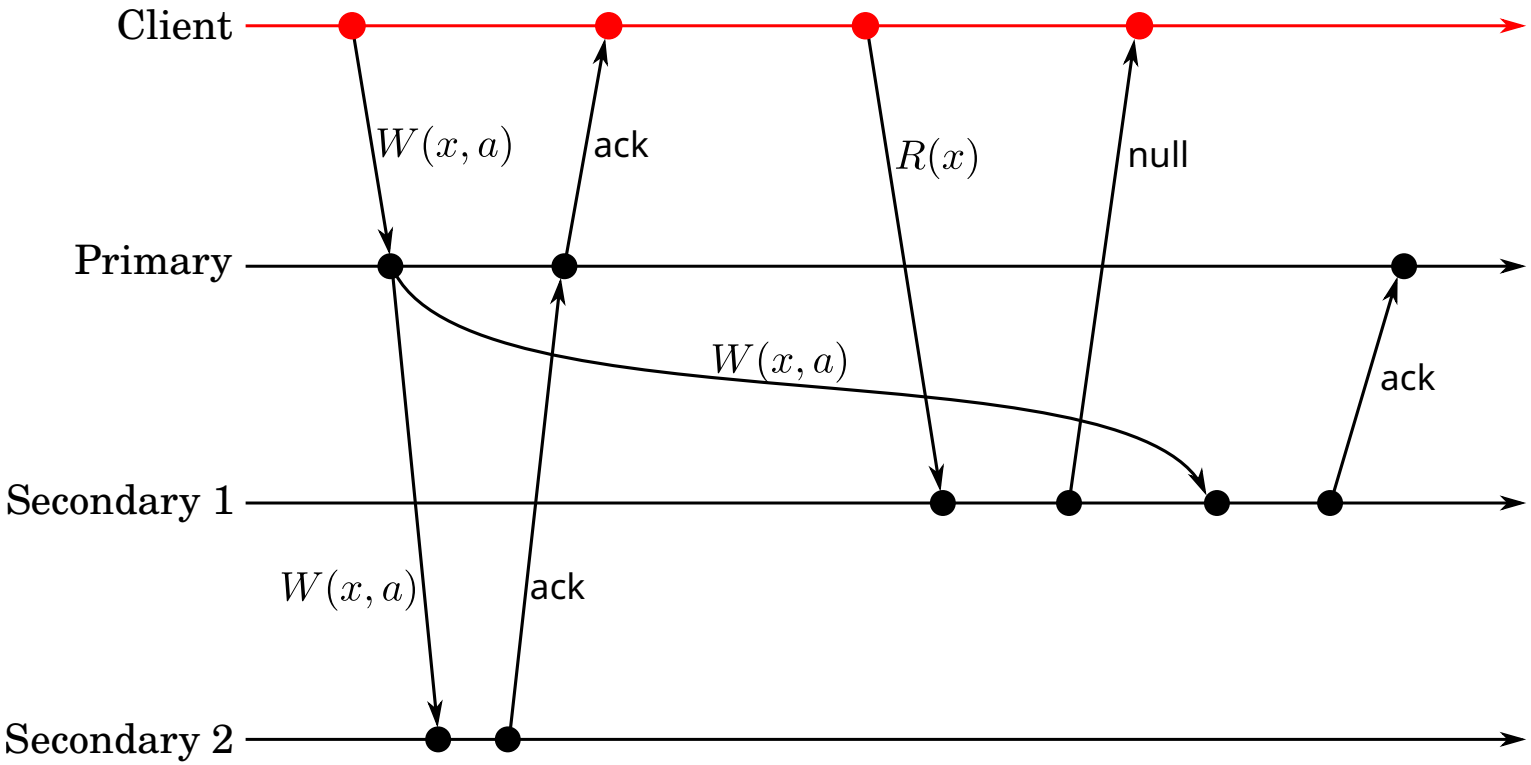
- The primary waits until the new value has been replicated to a majority of members.
- It then advances the majority-committed snapshot and acknowledges the write to the client.

Read concern "majority":

- The query reads from the majority-committed snapshot, not necessarily from the latest snapshot.
- In the previous example it returns $x = 3$, even if the node's latest snapshot contains $x = 4$ or $x = 5$.

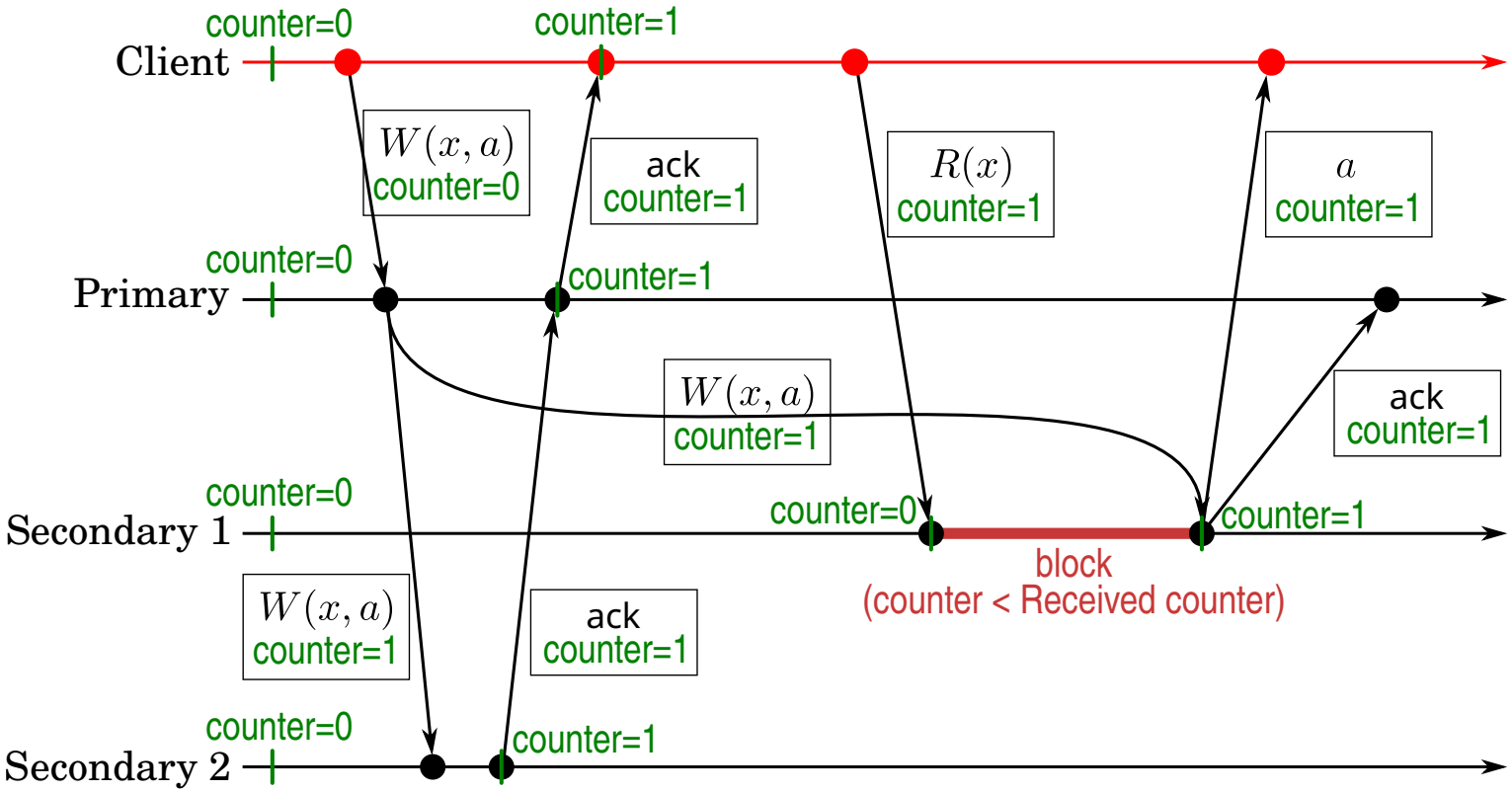
Implementation of read-your-writes consistency

Setting both read and write concerns to `"majority"` is not enough to guarantee read-your-writes consistency by definition:



Implementation of read-your-writes consistency

MongoDB resolves this problem by introducing Lamport clocks. The following conceptual diagram illustrates the idea:



For more details, see `https://dl.acm.org/doi/10.14778/3352063.3352125`

Tunable consistency and consistency levels

Apache Cassandra:



Tunable Consistency

Cassandra supports a per-operation tradeoff between consistency and availability through **Consistency Levels**. Cassandra's consistency levels are a version of Dynamo's `R + W > N` consistency mechanism where operators could configure the number of nodes that must participate in reads (`R`) and writes (`W`) to be larger than the replication factor (`N`). In Cassandra, you instead choose from a menu of common consistency levels which allow the operator to pick `R` and `W` behavior without knowing the replication factor. Generally writes will be visible to subsequent reads when the read consistency level contains enough nodes to guarantee a quorum intersection with the write consistency level.

`https://cassandra.apache.org/doc/4.1/cassandra/architecture/dynamo.html`

Tunable consistency and consistency levels

ScyllaDB:



Consistency Level

The **Consistency Level** (CL) determines how many replicas in a cluster must acknowledge a read or write operation before it is considered successful.

Some of the most common Consistency Levels used are:

- **ANY** – (Write Only) – If all replica nodes are down, write succeeds after a hinted-handoff. Provides low latency, guarantees writes never fail.
- **QUORUM** – When a majority of the replicas respond, the request is honored. If RF=3, then 2 replicas respond. QUORUM can be calculated using the formula *(n/2 +1)* where *n* is the Replication Factor.
- **ONE** – If one replica responds; the request is honored.
- **LOCAL_ONE** – At least one replica in the local data center responds.
- **LOCAL_QUORUM** – A quorum of replicas in the local datacenter responds.
- **EACH_QUORUM** – *(unsupported for reads)* – A quorum of replicas in ALL datacenters must be written to.
- **ALL** – A write must be written to *all* replicas in the cluster, a read waits for a response from *all* replicas. Provides the lowest availability with the highest consistency.

https://university.scylladb.com/courses/scylla-essentials-overview/lessons/high-availability/topic/consistency-level/

Consistency levels in ScyllaDB

In ScyllaDB, the client sends all *Read* and *Write* requests to the **coordinator node**:

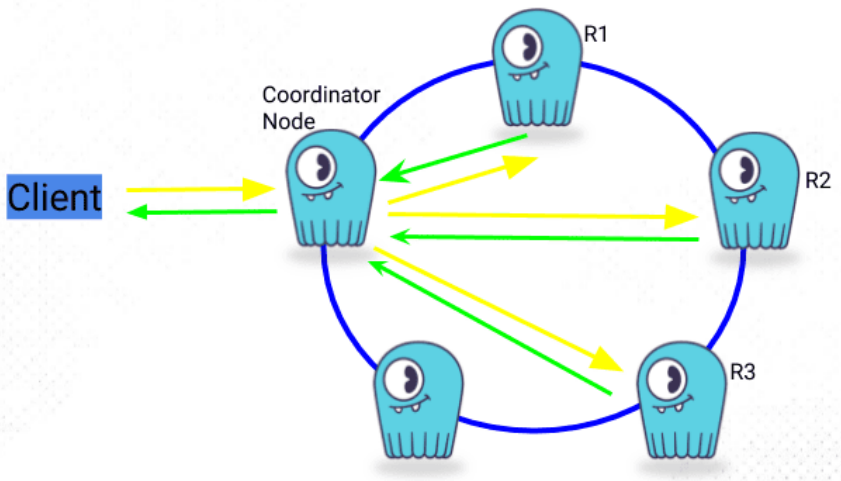


Figure from `https://university.scylladb.com/courses/scylla-essentials-overview/lessons/architecture/topic/cluster-level-read-write-interaction/`

The coordinator node will forward the requests to the **replica nodes** and process the requests according to the **replication factor (RF)** and the **consistency level (CL)**.

Consistency levels in ScyllaDB

Examples:

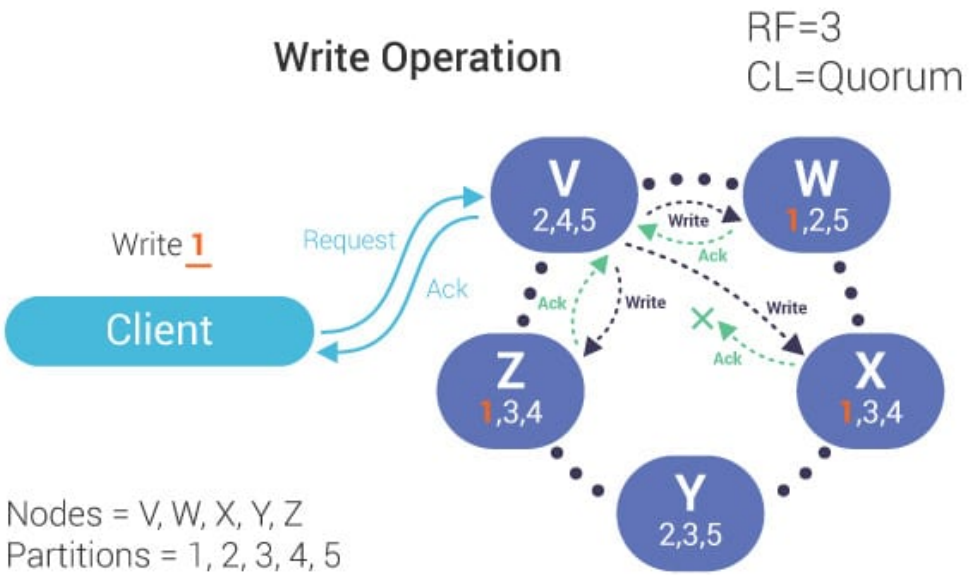


Figure from `https://docs.scylladb.com/manual/stable/architecture/architecture-fault-tolerance.html`

Consistency levels in ScyllaDB

Examples:

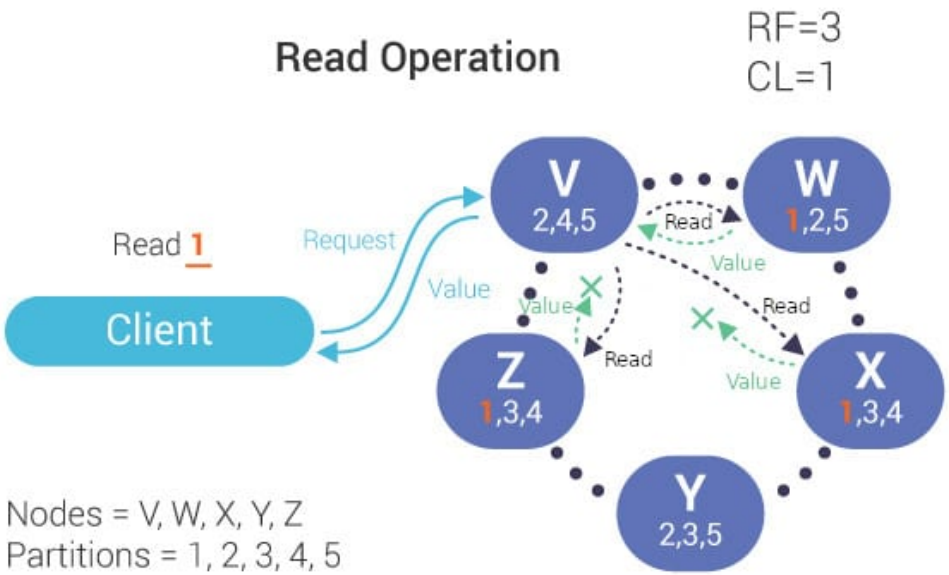


Figure from `https://docs.scylladb.com/manual/stable/architecture/architecture-fault-tolerance.html`

Digest reads and read repair

To reduce network traffic, the coordinator does not initially request the full row from every replica.

$$\begin{aligned} \text{Coordinator} &\longrightarrow \text{Replica 1: request full data} \\ &\longrightarrow \text{Replica 2: request digest} \\ &\longrightarrow \text{Replica 3: request digest} \end{aligned}$$

A digest is a compact hash of the requested data.

- If the digests agree, the replicas contain the same data and the coordinator returns the full-data response to the client.
- If the digests disagree, the coordinator requests the full data from the replicas and reconciles the versions, usually using timestamps.
- The newest version is returned to the client and sent to stale replicas. Updating those replicas is called read repair.

digest mismatch $\Longrightarrow$ fetch full versions $\Longrightarrow$ reconcile and repair

1 Motivation

2 Data-centric consistency models

3 Client-centric consistency models

4 Summary

Summary

Summary:

- Data-centric consistency models:
  - Strict consistency (linearizability)
  - Sequential consistency
  - Causal consistency
  - Eventual consistency
- Implementations of some consistency models using FIFO-total order broadcast.
- Client-centric consistency models: Read-your-writes, write-follow-reads, monotonic reads, monotonic writes.